

Software Bug Prediction Employing Feature Selection and Deep Learning

Samar M. Abozeed
Computer and Systems
Engineering

Alexandria University
Alexandria, Egypt
samar.abo.zeed@gmail.com

Mustafa Y. ElNainay
Faculty of Computer and
Information Systems
Islamic University

AlMadinah AlMunnawarah, KSA
melnainay@iu.edu.sa

Soheir A. Fouad
Computer and Systems
Engineering

Alexandria University
Alexandria, Egypt
saf@alexu.edu.eg

Mohamed S. Abougabal
Computer and Systems
Engineering

Alexandria University
Alexandria, Egypt
mohamed.abougabal@alexu.edu.eg

Abstract—It was proven that the cost of fixing errors escalates as a project moves through its life cycle in an exponential fashion. Identifying buggy classes, as soon as they are committed to the Version Control System, would have a significant impact on reducing such cost. Mining in software repositories is a growing research area, where innovative techniques and models are designed to analyze software repositories data and uncover useful information that can help in software bug prediction. Previous studies showed that Deep Learning has achieved remarkable results in many fields and it keeps evolving.

In this paper, experiments are carried out to study the effect of feature selection on the performance of bug prediction models and to check if better results can be obtained by using the promising Deep Learning techniques. Results show that applying feature selection, using a simple filter approach, such as selecting the highly ranked 9 and 5 features out of the 17 features, did not enhance the performance measures in most cases. On the other hand, results show that Deep Learning model (DL) achieves higher performance measures than the selected set of base classifiers for small and balanced datasets.

Keywords—Bug Prediction, Defect Prediction, Change Metrics, Feature Selection, Deep Learning.

I. INTRODUCTION

The majority of software organizations are small to medium enterprises that experience limited resources and strict deadlines. Under these limitations producing high quality products with the lowest cost becomes very challenging [1]. Testing and bug fixing phases are used to maintain the product quality and it was shown that the escalation of the cost to fix errors as a project moves through its life cycle increases in an exponential fashion [2]. There is no doubt that identify buggy classes as soon as they are committed to the Version Control System (VCS) would have a significant impact on reducing the fixing cost, and hence the product cost.

The primary contribution of this paper is to extend the work in [3] by considering larger datasets, feature tuning by applying feature selection techniques and considering the use of Deep Learning for the classification. In [3], multiple models were built and evaluated based on weighted majority voting ensemble techniques [4-6] such as Weighted Majority Voting (WM), Cascaded Weighted Majority Voting (CWM), Randomized Weighted Majority Voting (RWM), and Cascaded Randomized Weighted Majority Voting (CRWM). Static code metrics, change metrics, and combined metrics of both have been used as input features to the classifiers. The study

concluded that the best classification performance can be reached using change metrics and CRWM technique.

The remaining of this paper is organized as follows. In Section II, the related work on different bug prediction techniques and used software metrics are presented. In Section III, the proposed bug prediction model is explained in details. In Section IV, the used datasets, performance measures and performance evaluation are described. Finally, the paper is concluded in Section V.

II. RELATED WORK

One of the major fields that can identify buggy classes as soon as they are committed to the Version Control system (VCS) is mining in software repositories. Mining software data along with their related challenges can be found in a number of surveys in the literature [7-11].

Software bug identification is categorized as supervised learning, for that all provided instances must be classified first as buggy or clean before input them to the classification techniques to build the models. Since most bug prediction models are created based on the existing software source code, where data are not labeled for that reason, the first step is to label the input data to either buggy or clean. Data Labeling is one of the most challenging steps for any bug prediction technique. There are multiple of suggested algorithms used for data labeling [12-14] that have been introduced to tackle that challenge.

Second step is to extract useful features that could help in bug prediction, these are called software metrics. The existing software metrics are divided mainly into two categories: product-related (static code metrics) and process-related metrics (change metrics). These two categories also could be combined to form a new hybrid set of metrics. All of these metrics have been either manually designed, formed by selecting combinations from the already existing sets [15-18] or dynamically extracted from the source code [19, 20]. Feature selection algorithms [21, 22] are addressed to reduce the redundant and irrelevant features by selecting only the subsets that best describe the model to increase the model performance. A number of methods are used in feature selection algorithms surveyed in [23].

Third step is to select a technique for building the model. Many machine learning techniques have been employed in bug prediction models [24] such as Naïve Bayes (NB), Logistic Regression (LR), Decision Trees (DT), Support

Vector Machine (SVM), and ensemble techniques that showed remarkable performance over single base classifiers [25-27]. A model was suggested in [3] that employed Weighted Majority Voting technique achieves better results. Deep Learning [28] is a promising potential technique for this step as it has been evolving and has achieved remarkable results in many fields such as natural language processing, image processing, among other fields [29], also used for spam detection and malicious domain names detection [30-32].

Fourth and final step is to use the developed software bug prediction model to identify whether a newly committed change has potential bugs or not. The work presented in this paper extends the work done in [3] to enhance the performance of bug prediction using feature selection and deep learning.

III. THE PROPOSED SOFTWARE BUG PREDICTION MODEL

The proposed model uses more instances by considering more revisions than used in [3]. The impact of feature selection [21, 22] on the performance measures was examined. In addition, a Deep Learning model (DL) is implemented. A comparative study has been performed and the results are in the Datasets and Evaluation Environment section. The proposed bug prediction model is based on three main steps: Attributes (Metrics) retrieval, Feature selection and Bug prediction models as depicted in Fig. 1.

A. Attributes (Metrics) Retrieval

In this step, software metrics are extracted from the software data and used as the input to the bug prediction models. It consists of two sub-steps.

1) *Data Labeling*: the SZZ algorithm [12] is used to label the data. SZZ algorithm is called after the first letter of the authors' last name (Jacek Sliwerski, Thomas Zimmermann, Andreas Zeller), the algorithm labels the data by creating a link between the version control system (VCS) commits and bugs reported in the bug tracking systems. The basic idea is listed in the points below:

- First, a report is generated from the Bug Tracking System by selecting all bugs that were marked as FIXED.
- Get the committed change logs from the VCS and link it with the report extracted in the previous step, this way fix locations for the reported fixed bugs are identified.
- Search for the earlier change for each location that have been applied before the bug was reported to identify the bug introducing change.

More revised versions from the SZZ were introduced in [13] that have an enhanced performance in decreasing the false positives and false negatives by using: annotation graphs, ignoring any non-semantic source code changes and outlier fixes. They removed about 38%~51% of the false positives and 14%~15% of the false negatives. Another work was presented in [14] that suggests improvements to the SZZ algorithm by replacing the annotation graph and use line number maps, using a DiffJ tool to ignore comments and formatting changes. In this paper, the same version of the SZZ used in [3] is used.

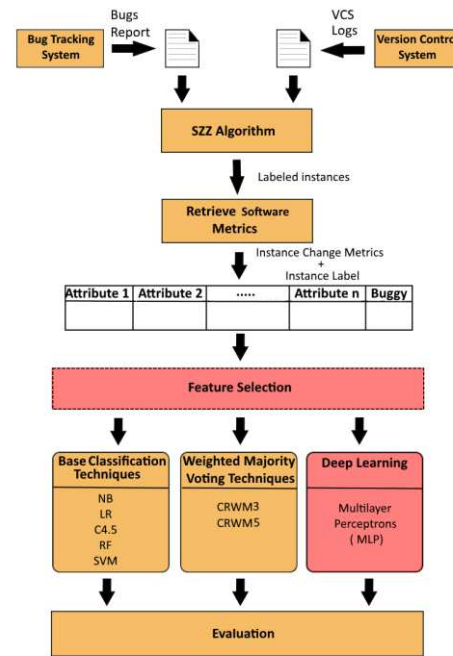


Fig. 1. Software Defect Prediction Workflow

2) Software Metrics

The first type of metrics is the static code metrics, also known as product-related metrics, which extracts useful information about the code. The second type is the change metrics, also known as process-related metrics, and this type of metrics requires historical information. It has been shown that change metrics have better performance [15] and predictability accuracy relative to the static code metrics [33]. The third type is a combination from the product-related and process-related metrics. In the proposed work, the change metrics performance will be examined as it was proved to have better performance in [3] and will leave the static code metrics and the combined metrics for future work. TABLE I, lists all change metrics used in this paper, same as those used in [3] as we are comparing results with it.

B. Feature selection

Selecting relevant and non-redundant features (metrics) may improve the model performance. A number of methods are used for feature selection surveyed in [23]. The most popular approaches used for feature selection are filter and wrapper approaches.

In the filter approach, the feature selection technique is independent of the used data mining model (Prediction model in our case) and it evaluates the importance of features based on the properties of data, the output of selected features is then supplied to the data mining algorithm. In the wrapper approach, the feature selection technique is integrated with the data mining model which gives the advantage of considering feature dependencies. Quality of wrapper approach depends on the data mining. In this paper, the filter approach is used. Other alternative approaches are left for future work.

TABLE I. LIST OF CLASS LEVEL CHANGE FEATURES

Feature	Meaning
Revisions	Number of file revisions
Refactoring	Number of file refactoring times
Bug Fixes	Number of bug-fixing times of a file
Authors	Number of file distinct authors
Added LOC	Total added lines over all file revisions
Max Added LOC	The largest added LOC over all file revisions
AVE Added LOC	Average added LOC per revision
Deleted LOC	Total deleted lines over all file revisions
Max Deleted LOC	The largest deleted LOC over all file revisions
AVE Deleted LOC	Average deleted LOC per revision
CODECHURN	(Added – deleted) lines over all file revisions
Max CODECHURN	The largest CODECHURN over all file revisions
AVE CODECHURN	Average of CODECHURN per revision
Max CHANGESET	The largest number of files committed together
AVE CHANGESET	Average number of files committed together
Age	File age in weeks measured from a specific revision
Weighted Age	$\frac{\sum_{i=1}^N Age(i) \times LOC_Added(i)}{\sum_{i=1}^N LOC_Added(i)}$

C. Bug Prediction Models

The data produced from the data retrieval step, with and without feature selection, forms the input to four base classifiers (NB, LR, C4.5 and SVM), one ensemble classifier (Random Forest (RF)), two Weighted Majority Voting techniques (CRWM3, CRWM5) and a Deep Learning model (DL). CRWM3 is an ensemble of three classifiers namely (NB, LR, C4.5), and CRWM5 is an ensemble of five classifiers namely (NB, LR, C4.5, SVM and RF).

A number of general categories are widely used in Deep Learning [28]. Feedforward Networks also known as Multilayer Perceptron (MLP), Convolution Neural Network (CNN), and Recurrent Neural Network (RNN) are famous examples of Deep learning models. The different Deep learning models usually follow the structure of the data. E.g. MLP with fully connected layers is usually used when the model presents supervised learning with fixed input vector. CNN is usually used when the input has a known topology e.g. images. RNN is usually used when the input is a sequence. Since the bug prediction model is a supervised learning with fixed input vector. MLP model with fully connected layers approach is used in our implementation.

IV. DATASETS AND EVALUATION ENVIRONMENT

In this section, the used datasets are introduced and the evaluation results between various techniques are presented.

A. Data collection

The datasets used were based on the work in [3]. As a proof of concept, three datasets are selected (Log4j, JMeter and Ant) from the four datasets used in [3]. The fourth dataset (Tomcat7) is left for future work. All datasets are Apache open source java projects that can be downloaded from

Apache website [34]. TABLE II lists the datasets used in this work. It is observed from TABLE II that Log4j and JMeter datasets have buggy records around 45% in such case these two databases are almost balanced datasets whereas the buggy records for the Ant dataset is around 18% which represents an imbalanced dataset. More data instances are considered compared to the work in [3] by including all VCS submitted revisions. Due to this increase, the classification results, reported in [3], have changed. A comparison between the number of instances, used in [3] and in the proposed work in this paper is listed in TABLE III. Moreover, the following combinations, outlined in TABLE IV, are used to study and evaluate balanced and imbalanced datasets with both small and large number of instances.

TABLE II. DATASET DESCRIPTION

Project	Software Type	Revision Number	Class Distribution Buggy: Clean	Buggy Rate (%)
Apache Log4j	Logging framework in java	1683591	804:999	44.59
Apache JMeter	Load testing tool for analyzing and measuring the performance	1842697	10858:12324	46.84
Apache Ant	Software tool for automating software build processes	1592318	6485:28853	18.35

TABLE III. DIFFERENCE IN INSTANCES NUMBER AFTER SUPPORTING ALL REVISIONS

Project	Number of instances [3]	Number of instances (Proposed model)
Apache Log4j	407	1803
Apache JMeter	1359	23182
Apache Ant	2052	35338

TABLE IV. DATASETS USED FOR EVALUATION

	Small datasets	Large datasets
Balanced datasets	-Apache Log4j -Subset of Apache JMeter	-Apache JMeter
Imbalanced datasets	-Subset of Apache Ant	-Apache Ant

B. Performance Measures

To evaluate the predictability performance of the software bug prediction models, performance measures that are commonly used in literature are selected: F-Measure (FM), also known as F1 score, Accuracy (AC) and Area under the ROC curve (AUC). The F-Measure is the harmonic mean between precision and recall. F-Measure is best used in case of imbalanced data. The Accuracy is the widely used performance measure for classification problems but it has been verified that it could be a poor performance measure in case of imbalanced data. The area under the ROC curve is also adopted. ROC curve shows the trade-off between goodness e.g. true positive rate (TPR) and cost e.g. false positive rate (FPR). All mathematical definitions for the performance measures used are listed in [3].

C. Parameter Settings for Training Bug Prediction Models

In this subsection, all selected values related to different models are listed. For each dataset, instances are divided to 20% as validation set, which is used to extract model parameters, and the remaining 80% are used as input for cross-validation with 10 folds [35].

1) *Base, ensemble and Weighted Majority Voting Techniques Settings*: The modified version from the Weka open source tool used in [3], where Weighted Majority Voting techniques were implemented, is used to extract performance measures. Weka's parameters are selected as suggested in [3].

2) *Features (Metrics) Selection*: for the conducted experiments, the highly ranked 9 and 5 features, listed in TABLE V and TABLE VI, are extracted using scikit library (A filter approach method, SelectKBest, is utilized while using chi2 as the scoring function [36]). The same extracted set of features are applied to all techniques. More feature selection approaches, methods and different number of selected attributes are left as a future work.

TABLE V. SELECTED BEST 9 ATTRIBUTES

Feature	Log4j	Small JMeter	Small Ant	JMeter	Ant
Revisions			✓		✓
Bug Fixes		✓			
Added LOC	✓	✓	✓	✓	✓
Max Added LOC	✓	✓	✓	✓	✓
AVE Added LOC					
Deleted LOC	✓	✓	✓	✓	✓
Max Deleted LOC		✓		✓	
CODECHURN	✓		✓	✓	✓
Max CODECHURN	✓	✓	✓	✓	✓
Max CHANGESET	✓			✓	
AVE CHANGESET	✓	✓	✓		✓
Age	✓	✓	✓	✓	✓
Weighted Age	✓	✓	✓	✓	✓

TABLE VI. SELECTED BEST 5 ATTRIBUTES

Feature	Log4j	Small JMeter	Small Ant	JMeter	Ant
Added LOC	✓		✓	✓	✓
Deleted LOC	✓	✓	✓	✓	✓
Max Deleted LOC		✓			
AVE Deleted LOC					
CODECHURN	✓		✓	✓	✓
Max CHANGESET				✓	
AVE CHANGESET		✓			
Age	✓	✓	✓		✓
Weighted Age	✓	✓	✓	✓	✓

3) *Deep Learning Model Settings*: A Deep Learning model (DL) is implemented, using scikit machine learning library [36] on the Scientific Python Development Environment IDE (Spyder) under Anaconda package manager [37]. The validation datasets are used to extract the

hyperparameter values needed for the DL model. In order to simplify the search, the same number of hidden units are used in each hidden layer. For all datasets, Adam is used as the optimization algorithm and ReLU activation function is used for all units except the output layer unit where sigmoid activation is used.

For Log4j dataset, 8 layers, 32 units in each hidden layer, 150 iterations and 0.01 learning rate are used while employing 1024 batch size when no feature selection is applied and 2048 when feature selection is applied. For Small JMeter dataset, 4 layers, 64 units in each hidden layer, 32 batch size and 0.001 learning rate are used while employing 200 iterations when no feature selection is applied and 150 when feature selection is applied. For Small Ant dataset, 5 layers, 64 units in each hidden layer, 150 iterations, 64 batch size and 0.001 learning rate are used while employing 20% dropout rate.

For JMeter dataset, 5 layers, 64 units in each hidden layer, 200 iterations, 32 batch size and 0.001 learning rate are used. For Ant dataset, 5 layers, 64 units in each hidden layer, 200 iterations, 64 batch size and 0.001 learning rate are used.

D. Performance Evaluation

For each dataset in TABLE IV, experiments were conducted for: NB, LR, C4.5, SVM, RF, CRWM3, CRWM5 and MLP Deep Learning model (DL). Best performance is indicated in **bold** while the second-best is underlined.

It is observed from results in TABLE VII to TABLE XI that performance measures, in most cases, are slightly degraded after applying feature selection. Moreover, implementing Deep Learning as a classifier further enhanced the performance measures when applied on small balanced datasets.

The performance measures have slightly enhanced for Deep Learning on large balanced dataset relative to its small subset. Nevertheless, more investigation is needed to study the performance of Deep Learning on large balanced datasets as well as small and large imbalanced datasets. Moreover, more experiments need to be conducted to examine if further hyperparameter tuning or, in case of imbalanced datasets, using oversampling, under-sampling and/or changing the loss function to be more sensitive to the minority class, would enhance the performance measures.

TABLE VII. LOG4J RESULTS WITH DIFFERENT TECHNIQUES

Log4j	Change Metrics								
	No Feature Selection			With 9 Selected Features			With 5 Selected Features		
	FM	AC	AUC	FM	AC	AUC	FM	AC	AUC
NB	0.60	60.89	0.70	0.58	60.33	0.70	0.53	57.84	0.71
LR	0.65	65.81	0.72	0.67	67.13	0.72	0.65	65.60	0.70
C4.5	0.69	69.00	0.72	0.66	65.81	0.72	0.68	67.75	0.70
SVM	0.61	63.18	0.61	0.58	61.51	0.58	0.56	60.61	0.57
RF	<u>0.73</u>	<u>72.75</u>	0.79	<u>0.73</u>	72.88	<u>0.80</u>	0.70	69.97	0.77
CRWM3	0.71	70.39	<u>0.77</u>	0.69	69.56	<u>0.80</u>	<u>0.75</u>	<u>75.10</u>	<u>0.85</u>
CRWM5	0.72	71.91	0.79	0.77	76.70	0.83	0.84	83.84	0.88
DL	0.74	74.15	0.74	<u>0.73</u>	<u>73.77</u>	0.73	0.71	71.83	0.71

TABLE VIII. SMALL JMETER RESULTS WITH DIFFERENT TECHNIQUES

	Change Metrics								
	No Feature Selection			With 9 Selected Features			With 5 Selected Features		
	FM	AC	AUC	FM	AC	AUC	FM	AC	AUC
NB	0.57	57.00	0.59	0.53	54.44	0.60	0.53	54.79	0.57
LR	0.64	63.80	0.70	0.60	60.40	0.66	0.55	56.87	0.61
C4.5	0.66	66.30	0.70	<u>0.65</u>	65.26	0.68	0.65	65.67	0.71
SVM	0.62	62.97	0.61	0.60	60.61	0.59	0.51	56.03	0.53
RF	<u>0.71</u>	<u>71.29</u>	0.78	0.72	<u>71.98</u>	0.78	<u>0.70</u>	<u>70.25</u>	0.76
CRWM3	0.64	64.29	0.70	<u>0.65</u>	64.63	0.70	0.66	66.09	0.68
CRWM5	0.65	65.46	<u>0.74</u>	<u>0.65</u>	65.33	<u>0.73</u>	0.63	64.08	<u>0.72</u>
DL	0.73	73.57	0.73	0.72	72.15	0.72	0.72	72.23	<u>0.72</u>

TABLE IX. SMALL ANT RESULTS WITH DIFFERENT TECHNIQUES

	Change Metrics								
	No Feature Selection			With 9 Selected Features			With 5 Selected Features		
	FM	AC	AUC	FM	AC	AUC	FM	AC	AUC
NB	0.81	81.00	0.80	0.82	81.76	0.82	0.81	81.14	0.80
LR	0.84	84.74	0.87	0.83	84.60	0.84	0.82	84.19	0.80
C4.5	<u>0.87</u>	87.24	0.80	<u>0.86</u>	86.48	0.80	<u>0.85</u>	85.99	0.76
SVM	0.81	84.19	0.62	0.79	83.29	0.59	0.73	80.79	0.52
RF	0.88	<u>88.84</u>	0.91	0.88	88.42	0.90	0.87	<u>87.52</u>	0.88
CRWM3	<u>0.87</u>	87.03	0.87	0.85	86.13	0.85	0.84	84.95	0.79
CRWM5	0.88	88.42	<u>0.89</u>	<u>0.86</u>	87.52	<u>0.88</u>	<u>0.85</u>	86.48	<u>0.85</u>
DL	0.84	90.14	0.82	0.79	<u>88.10</u>	0.77	0.78	88.10	0.74

TABLE X. JMETER RESULTS WITH DIFFERENT TECHNIQUES

	Change Metrics								
	No Feature Selection			With 9 Selected Features			With 5 Selected Features		
	FM	AC	AUC	FM	AC	AUC	FM	AC	AUC
NB	0.58	58.41	0.63	0.58	59.19	0.65	0.56	57.47	0.62
LR	0.64	63.74	0.68	0.61	61.44	0.65	0.60	60.35	0.63
C4.5	0.73	73.15	0.76	0.71	71.15	0.76	0.66	66.14	<u>0.71</u>
SVM	0.64	63.93	0.64	0.62	62.41	0.62	0.56	56.86	0.57
RF	0.77	76.68	0.84	0.76	75.97	0.83	0.71	70.59	0.78
CRWM3	0.69	68.57	0.75	0.69	68.79	0.75	0.64	63.91	0.69
CRWM5	0.68	68.28	<u>0.78</u>	0.69	68.68	<u>0.77</u>	0.63	62.98	<u>0.71</u>
DL	<u>0.76</u>	<u>75.61</u>	0.76	<u>0.74</u>	<u>73.52</u>	0.74	<u>0.70</u>	<u>70.46</u>	0.70

TABLE XI. ANT RESULTS WITH DIFFERENT TECHNIQUES

	Change Metrics								
	No Feature Selection			With 9 Selected Features			With 5 Selected Features		
	FM	AC	AUC	FM	AC	AUC	FM	AC	AUC
NB	0.80	80.36	0.76	0.79	80.00	0.76	0.79	80.16	0.74
LR	0.81	83.53	0.82	0.80	83.23	0.77	0.79	83.06	0.76
C4.5	<u>0.86</u>	86.11	0.82	<u>0.85</u>	85.77	0.84	<u>0.83</u>	84.63	0.82
SVM	0.79	83.11	0.59	0.78	82.77	0.57	0.77	82.35	0.55
RF	0.87	<u>87.90</u>	0.92	0.87	<u>87.72</u>	0.92	0.86	<u>86.73</u>	0.89
CRWM3	0.83	84.43	0.84	0.82	83.93	0.82	0.81	83.66	0.79
CRWM5	0.82	84.73	<u>0.90</u>	0.82	84.61	<u>0.88</u>	0.81	84.37	<u>0.86</u>
DL	0.80	88.18	0.80	0.79	87.83	0.78	0.77	86.94	0.75

V. CONCLUSIONS AND FUTURE WORK

In this paper, software bug prediction models employing features selection and deep learning were studied. Results showed that applying feature selection, using a simple filter approach, such as selecting the highly ranked 9 and 5 features out of the 17 features, did not enhance the performance measures in most cases. Results also showed that Deep Learning model achieved higher performance measures than the selected set of base classifiers for small and balanced datasets.

Future work of the study introduced in this paper may include further tuning to the Deep Learning model, examining other deep learning models like CNN and RNN and investigating the performance of Deep Learning on large balanced datasets as well as small and large imbalanced datasets by employing oversampling, under-sampling and/or changing the loss function to be more sensitive to the minority class. In addition, studying the performance of the Deep Learning model on static and combined metrics while testing on more datasets with different sizes may be considered.

REFERENCES

- [1] Mishra, Deepti, and Alok Mishra. "Software process improvement in SMEs: A comparative view." *Comput. Sci. Inf. Syst.* 6, no. 1 (2009): 111-140.
- [2] Stecklein, Jonette M., Jim Dabney, Brandon Dick, Bill Haskins, Randy Lovell, and Gregory Moroney. "Error cost escalation through the project life cycle." *International Council on Systems Engineering 14th Annual International Symposium*, Toulouse; France, 19-24 June 2004.
- [3] Moustafa, Sammar, Mustafa Y. ElNainay, Nagwa El Makky, and Mohamed S. Abougabal. "Software bug prediction using weighted majority voting techniques." *Alexandria engineering journal* 57, no. 4 (2018): 2763-2774.
- [4] Littlestone, Nick, and Manfred K. Warmuth. "The weighted majority algorithm." *Information and computation* 108, no. 2 (1994): 212-261.
- [5] Zamani, Mohammadzaman, Hamid Beigy, and Amirreza Shaban. "Cascading randomized weighted majority: A new online ensemble learning algorithm." arXiv preprint arXiv:1403.0388 (2014).
- [6] Rokach, Lior. "Ensemble methods for classifiers." *In Data mining and knowledge discovery handbook*, pp. 957-980. Springer, Boston, MA, 2005.
- [7] Kagdi, Huzefa, Michael L. Collard, and Jonathan I. Maletic. "A survey and taxonomy of approaches for mining software repositories in the context of software evolution." *Journal of software maintenance and evolution: Research and practice* 19, no. 2 (2007): 77-131.
- [8] Anbalagan, Prasanth, and Mladen Vouk. "On mining data across software repositories." *In 2009 6th IEEE International Working Conference on Mining Software Repositories*, pp. 171-174.
- [9] Xie, Tao, Suresh Thummalapenta, David Lo, and Chao Liu. "Data mining for software engineering." *Computer* 42, no. 8 (2009): 55-62.
- [10] Halkidi, Maria, Diomidis Spinellis, George Tsatsaronis, and Michalis Vazirgiannis. "Data mining in software engineering." *Intelligent Data Analysis* 15, no. 3 (2011): 413-441.
- [11] D'Ambros, Marco, Harald Gall, Michele Lanza, and Martin Pinzger. "Analysing software repositories to understand software evolution." *In Software evolution*, pp. 37-67. Springer, Berlin, Heidelberg, 2008.
- [12] Śliwinski, Jacek, Thomas Zimmermann, and Andreas Zeller. "When do changes induce fixes?." *In ACM sigsoft software engineering notes*, vol. 30, no. 4, pp. 1-5. ACM, 2005.
- [13] Kim, Sunghun, Thomas Zimmermann, Kai Pan, and E. James Jr. "Automatic identification of bug-introducing changes." *In 21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*, pp. 81-90. IEEE, 2006.

- [14] Williams, C. and Spacco, J., 2008, July. Szz revisited: verifying when changes induce fixes. *In Proceedings of the 2008 workshop on Defects in large software systems* (pp. 32-36). ACM.
- [15] Moser, Raimund, Witold Pedrycz, and Giancarlo Succi. "A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction." *In Proceedings of the 30th international conference on Software engineering*, pp. 181-190. ACM, 2008.
- [16] Giger, Emanuel, Marco D'Ambros, Martin Pinzger, and Harald C. Gall. "Method-level bug prediction." *In Proceedings of the ACM-IEEE international symposium on Empirical software engineering and measurement*, pp. 171-180. ACM, 2012.
- [17] Taba, Seyyed Ehsan Salamat, Foutse Khomh, Ying Zou, Ahmed E. Hassan, and Meiappan Nagappan. "Predicting bugs using antipatterns." *In 2013 IEEE International Conference on Software Maintenance*, pp. 270-279. IEEE, 2013.
- [18] Herzig, Kim, Sascha Just, Andreas Rau, and Andreas Zeller. "Predicting defects using change genealogies." *In 2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 118-127. IEEE, 2013.
- [19] Li, Jian, Pinjia He, Jieming Zhu, and Michael R. Lyu. "Software defect prediction via convolutional neural network." *In 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*, pp. 318-328. IEEE, 2017.
- [20] Dam, Hoa Khanh, Trang Pham, Shien Wee Ng, Truyen Tran, John Grundy, Aditya Ghose, Taeksu Kim, and Chul-Joo Kim. "A deep tree-based model for software defect prediction." arXiv preprint arXiv:1802.00921 (2018).
- [21] Guyon, Isabelle, and André Elisseeff. "An introduction to variable and feature selection." *Journal of machine learning research*, no.3, Mar (2003): 1157-1182.
- [22] Kohavi, Ron, and George H. John. "Wrappers for feature subset selection." *Artificial intelligence* 97, no. 1-2 (1997): 273-324.
- [23] Chandrashekar, Girish, and Ferat Sahin. "A survey on feature selection methods." *Computers & Electrical Engineering* 40, no. 1 (2014): 16-28.
- [24] Jing, Xiao-Yuan, Shi Ying, Zhi-Wu Zhang, Shan-Shan Wu, and Jin Liu. "Dictionary learning based software defect prediction." *In Proceedings of the 36th International Conference on Software Engineering*, pp. 414-423. ACM, 2014.
- [25] Dietterich, Thomas G. "Ensemble methods in machine learning." *In International workshop on multiple classifier systems*, pp. 1-15. Springer, Berlin, Heidelberg, 2000.
- [26] Neumayer, Robert. "Clustering based ensemble classification for spam filtering". *Proceedings of the 7th Workshop on Data Analysis (WDA'06)*, pp. 11-22, 2006.
- [27] Fan, Wei, and Salvatore J. Stolfo. "Ensemble-based adaptive intrusion detection." *In Proceedings of the 2002 SIAM International Conference on Data Mining*, pp. 41-58. Society for Industrial and Applied Mathematics, 2002.
- [28] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- [29] LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. "Deep learning." *nature* 521, no. 7553 (2015): 436.
- [30] Mi, Guyue, Yang Gao, and Ying Tan. "Apply stacked auto-encoder to spam detection." *In International Conference in Swarm Intelligence*, pp. 3-15. Springer, Cham, 2015.
- [31] Wu, Tingmin, Shigang Liu, Jun Zhang, and Yang Xiang. "Twitter spam detection based on deep learning." *In Proceedings of the australasian computer science week multiconference*, p. 3. ACM, 2017.
- [32] Vinayakumar, R., K. P. Soman, and Prabaharan Poornachandran. "Detecting malicious domain names using deep learning approaches at scale." *Journal of Intelligent & Fuzzy Systems* 34, no. 3 (2018): 1355-1367.
- [33] Rahman, Foyzur, and Premkumar Devanbu. "How, and why, process metrics are better." *In 2013 35th International Conference on Software Engineering (ICSE)*, pp. 432-441. IEEE, 2013.
- [34] Apache Subversion (LastAccess:2019). <<http://svn.apache.org/repos/asf/>>.
- [35] Zhang, Ping. "Model selection via multifold cross validation." *The Annals of Statistics* (1993): 299-313.
- [36] <https://scikit-learn.org> (LastAccess:2019).
- [37] Anaconda Distribution (LastAccess:2019). <<https://www.anaconda.com/distribution/>>